Two Pointers : Pair sum in a sorted array

Arr = [1, 5, 6, 10, 23, 43] , target = 15
0 1 2
Brute force : Naive solution
(Beginner - friendly solution)

Int n = 6; // size of array
(1, 5) -> 6
(1, 6) -> 7
(1, 10) -> 11
(1, 23) -> 24
(1, 43) -> 44
Number of pairs = n - 1 = 6 - 1 = 5

(5, 6) -> 11
(5, 10) -> 15
(5, 23) ->
(5, 43) ->

Number of pairs = n - 2 = 6 - 2 = 4

(6, 10) ->
(6, 23)->
(6, 43)->

Number of pairs = n - 3 = 6 - 3 = 3

(10, 23)->
(10, 43)->

Number of pairs = n - 4 = 2

(23, 43) ->

Number of pairs = n - 5 = 6 - 5 = 1

Total pairs = 15

N = 6, pairs = 15
Total pair of an array = (N * (N - 1)) / 2
= (N^2 - N) / 2

N = 100000
N ^ 2 = 10000000000
TC :

Optimize —-> Harder problem

Let’s optimize this brute force !!!

Two pointer algo :

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    // your code goes here

    // Two - pointer : Pair sum in a sorted array

    vector<int> a = {1, 5, 6, 10, 23, 43};
    int n = a.size();

    int target = 2;

    int i = 0;
    int j = n - 1;
    int flag = 0;
    while(i < j){
        int sum = a[i] + a[j];

        if(sum == target){
            flag = 1;
            cout << "Target found!!";
```

```
            break;
        }
        if(sum > target){
            j--;
        }else if(sum < target){
            i++;
        }
    }
    if(flag == 0) cout << "Target not found!!";

}
```

Sorting : Arranging elements in increasing order

[1, 5, 3, 10, 2] —-> after sorting result :[1, 2, 3, 5, 10]

There are many algorithms in sorting!!!

1) Selection Sort

[1, 5, 3, 10, 2] ->
Position of 1st Minimum element = 0
Position of 2nd Minimum element = 1
Position of 3rd Minimum element = 2
Position …….

[1, 2, 3, 4, 5] -> min = 5
 0 1 2 3 4

Key point : Select minimum element in an unsorted part of array, swap it into the next position.

Homework :
Task 1 : write brute force code of Pair sum in a sorted array

Task 2 : write two pointer : pair sum in a sorted array

Task 3 : write code of selection sort